

Project 3: Critters

1. Overview

Our goal in this assignment was to create an interpreter for a programming language that controls simulated creatures called critters. In this assignment, each critter belongs to a species and follows the instructions belonging to that species' behavior file. During simulation, critters can perform a variety of actions; to name a few, they can move, turn, inspect their surroundings, manipulate registers, infect enemies, and eat enemies if they're hungry. One unique aspect of this assignment is that a critter's behavior is executed over many turns. Therefore, the interpreter must remember each critter's state and resume from it during the next turn.

The primary task in this assignment was to implement the Interpreter class, which implements the provided CritterInterpreter interface. The class has two major responsibilities. Firstly, loadSpecies must read and validate behavior files before constructing any CritterSpecies objects. Secondly, executeCritter interprets a critter's loaded behavior code until it reaches an action instruction. Finally, the simulation performs the selected action.

We also created a custom test harness that allows us to control a critter's states and surroundings. This was more reliable than testing through the simulator mainly due to its randomness (this will be explained in more detail later). To end off the assignment, we also created our own critter, named Nalliah.

2. Description

We will now describe our implementation and solution in depth. To start, the loadSpecies method begins by opening the requested behavior file. The first line is read as the species name. Then, each following (nonblank) line is interpreted as an instruction, while the first blank line marks the end of the program. Afterwards, any text following that blank line is completely ignored, which allows for comments.

Each instruction is converted into an Argument object. An Argument stores the instruction type, its parameters, and an optional jump target. We presented instruction names with an Instruction enumeration rather than repeatedly comparing strings during execution. This reduces the chances of making spelling mistakes and makes the execution switch statement much easier to read and understand.

Something we had to consider was that different instructions accept different types and numbers of parameters. For example:

- the hop command accepts no parameters
- ifempty requires a bearing and target
- iflt requires two registers and a target.

To accommodate for this variability, we created a ParamFormat enumeration to associate each instruction with its required parameter structure. The parser uses this information to reject incorrect arguments, whether that be count or type.

Additionally, beyond validating count and type of parameters, the parser also validates the values of individual parameters. For instance:

- Registers must be named r1 through r10.
- Bearings must range from 0 through 315, and they must be multiples of 45
- Instruction or register names are converted to lowercase, which means they are not case sensitive

If an instruction cannot be parsed for whatever reason, we reject the behavior file all together instead of continuing to the execution stage.

Jump targets are slightly more complex, in that they support three target formats. A target without a sign is an absolute line number, a target beginning with +/- is relative to the current instruction, and a register target obtains an absolute line number from that register. To accommodate for these possibilities, we use a TargetType enumeration and a Target object. The getTrueLine method converts any of these representations into the actual next line at runtime. This means that execution logic does not need separate jump code for each conditional instruction.

The executeCritic method first obtains the critic's behavior code and saved line number through the provided Critic interface. Behavior code uses one-based line numbers, while Java uses zero-based indexing for lists. So, the interpreter must subtract one when retrieving an instruction from the list.

The interpreter then executes instructions in a loop. InWrite, inc, and go update the relevant state and allow execution to continue instead of ending the turn. Conditional instructions inspect the critic's hunger, nearby cells, headings, or register values. We created a shared check helper that either resolves the instruction's target when its condition is true or advances to the following instruction when it is false.

Action instructions are handled differently, however. When the interpreter reaches hop, left, right, eat, or infect, it calls the corresponding method on the critic, saves the following line with setNextCodeLine, and returns. The simulation engine is responsible for carrying out the

requested action. Ultimately, this separation follows the abstraction established by the provided API, as our interpreter chooses an action, but does not directly modify our simulated world.

If execution reaches a line outside the program, the critter must remain inactive for the remainder of the simulation. We represent this possibility using `code.size()+1`, which is positive and always invalid. This satisfies the Critter API but remains beyond the end of the program on every turn afterwards.

As for time complexity, `loadSpecies` is $O(n)$, where n is the amount of text in the behavior file. Its space usage is $O(m)$, where m is the number of behavior instructions stored. The `executeCritic` method is $O(k)$, where k is the number of instructions that are evaluated before an action or invalid line is reached. Each individual instruction requires constant time, $O(1)$. In most behavior files, k is relatively small, although a program containing an infinite loop could cause the interpreter to run infinitely long.

3. Discussion

Scope, Quality, Limitations

Our interpreter implements the full species language, with all five turn-ending actions, all nine conditional jumps, the five register instructions, and `go`. Every jumping instruction accepts all three target notations (absolute, signed relative, and register-indirect) resolved through a single `Target` object, with infect the deliberate exception since its parameter must be a plain line number. Behavior files are parsed once at load time into `Argument` objects, so `executeCritic` does no string processing. Parsing tolerates arbitrary whitespace and mixed-case keywords.

The result is correct and defensively written. Nothing throws as requested, and failures show as return values, with `loadSpecies` returning null and `executeCritic` returning quietly, and every rejection names the file and line. Validation runs before parsing, so `Integer.parseInt` never sees input that could make it fail.

The limitations are mostly deliberate trade-offs. We max execution at an honestly arbitrary 1000 instructions per turn, since nothing forces a program to reach an action and “go +0” would otherwise infinitely loop. We reject strings longer than nine digits, which is what guarantees `parseInt` cannot overflow without a `try/catch`, at the cost of excluding some valid int values. We do no analysis of loaded programs, and errors go to standard error, where a rejected species simply fails to appear in the setup list.

Interesting Results

The first concerns what "does nothing for the remainder of its turns" actually requires. Our initial `executeCritter` returned when a line fell out of range without storing that line back. For "go 99" this is effectively correct. But "ifenemy 0 99" followed by hop jumps out of bounds with an enemy ahead, leaves the stored line unchanged, and then hops on a later turn when no enemy is present, the critter comes back to life, which the specification forbids. Similarly, "incr1" followed by "go 99" increments forever instead of dying once. The fix was one line, but the bug is invisible in the simulator and was found by reasoning about the specification rather than by observation.

The second emerged only when we wrote our own critter. Because a turn ends the moment an action executes, the next turn resumes at the line after it, so a critter that infects on line 20 wakes on line 21 and blindly does whatever is there. This is why every provided species follows each action with "go 1". We initially overlooked these lines when they were actually important.

The third is that an infected critter inherits the new species' code but keeps its own registers, so no strategy can assume a register starts at zero. Our critter rewrites its zero constant every pass for this reason.

Assumptions

- Line numbers are 1-based; line n is element $n-1$, converted in exactly one place.
- Registers are numbered 1–10, matching `getReg/setReg` directly, and stored as written.
- Bearings are stored in degrees and passed unmodified to `getCellContent` and `getOffAngle`.
- `infect(n)` takes a 1-based line number; omitting it is equivalent to passing 1.
- `ifhungry` is true when hungry or starving; `ifstarving` only in the explicit starving state.
- Obstacles and the world edge both report as `WALL`, so `ifwall` covers both.
- `getOffAngle` returns `BAD` for an empty square, so `ifangle` needs no special case there.
- Keywords and register names are matched case-insensitively.
- The program ends at the first blank line; a file with an empty program is rejected.
- `loadSpecies` receives a path it can open directly.
- A critter running 1000 instructions without acting is malformed, and forfeiting its turn beats hanging the simulation.

Problems Encountered

Our most time-consuming problem was not a bug. After finishing `executeCriticter`, the world rendered but nothing moved. Printing the current line at the top of the execution loop produced no output, so the method was never being called. The real cause was that obstacles and critters are placed through separate paths, and obstacles render whether or not any critters exist. We had never clicked the button to create a simulation, and had mistaken a grid of stationary blocks for critters.

A second problem was structural. Our first parser signalled bad input by throwing `IOException`, which we had to remove once we learned exceptions were not permitted. Catching them was not enough, since `Integer.parseInt` throws on overflow. We restructured around validation-before-parsing: an `isInteger` helper confirms a token is a short run of digits before any conversion, and opcode lookup scans `values()` rather than calling `valueOf`. Each helper returns a sentinel or null and each caller checks immediately. The version is longer but has no path by which an exception could escape, rather than merely no path we anticipated.

We also found early that the simulator is a poor debugging method since placement and `ifrandom` are random, critters interact, register contents are invisible, and odd behavior gives no way to separate a bug from a weak strategy. That is what led to the separate test file described next in this report.

4. Testing

As per project instructions, we created a controlled test harness instead of relying on the inconsistent graphical simulator. The simulator, while useful for observing general behavior, is not ideal for robust testing. This is because of random placement, random behavior, and other critters. Another limitation of visual inspection is that we cannot see into internal states such as register values and the saved next instructions.

Our `TestCriticter` implements the provided `Criticter` interface and acts as a kind of mock simulation environment. That is, instead of moving through an actual world, it records which action the interpreter selected. The harness allows for each test to choose the critter's hunger level, nearby cells, off-angle, random result, register contents, and next code line. It also exposes the resulting action, register state, infection line, and saved instruction. We use temporary behavior files, which are created and used through JUnit's `@TempDir`, preventing test data from cluttering the project itself.

We have implemented 30 tests for the project. Rather than going through each individually, we will outline the major groups tested below:

- Species loading: this confirms that the species name and behavior code are loaded correctly, that comments after a blank line are ignored, and that an invalid bearing causes the file to be rejected.
- Action instructions: These tests individually test actions such as hop, left, right, eat, and infect, including the infection starting line.
- Jump instructions: Tests absolute, relative, and register-based target. It also verifies that jump outside the program leaves the critter inactive using a positive line that is out of the range (`code.size() + 1`).
- Conditional instructions: Tests `ifrandom`, `ifhungry`, `ifstartving`, `ifempty`, `ifally`, `ifenemy`, `ifwall`, and `ifangle` with controlled and specific inputs.
- Register instructions: Tests `iflt`, `ifeq`, and `ifgt` with register values that causes their jumps to occur.
- Register comparisons: Tests `iflt`, `ifeq`, and `ifgt` with register values that causes their jumps to occur.
- Execution state: Confirms that execution resumes after the previous turn's terminating action rather than just restarting from line one.
- Long execution: Uses more than 1,000 non-action instructions before a hop, which confirms that the interpreter allows a large amount of state manipulations before accepting an action.

Overall, our testing approach combines both black-box and white-box techniques. Checking which action was selected is an example of black-box as it examines the interpreter's externally observable result. On the other hand, checking register contents and the next saved lines is white-box testing as it verifies internal state. Together, these approaches test both the visible behavior of the interpreter and the logic needed to continue a program across multiple turns.

All 30 of our test cases pass.

5. Social Impact

CritterFest demonstrates both the benefits and drawbacks/risks of competition. On one hand, competition can motivate participants to experiment, optimize solutions, and learn from others in a way they cannot individually. This kind of competition has produced innovation in the real world, especially in capitalistic economies. On the other hand, however, competition can also encourage people to focus too much on winning at any cost, which can lead them to disregard ethics. In situations like gaming, this can contribute to toxicity. But in higher-stakes settings, this can give rise to unsafe technologies. Right now especially, AI alignment is a

prevalent issue because some believe AI is developing too fast: in hopes of developing the smartest model, labs have disregarded safety guardrails.

6. Pair Programming

This assignment was our first pair programming project in the class so far, so there was a lot to learn and get used to. We were largely successful in producing correct code. This is because we were able to effectively bounce ideas off each other. A second perspective can often expose assumptions or edge cases that one individual might overlook. This is relevant to work in the real world, whether that be industry or academia, where companies and research labs involve teams of professionals to achieve a goal. Team collaboration is different from working alone in many ways. For example, communication is vital in a team, as if partners do not understand each other's code and ideas, collaboration will be poor. When working alone, communication is less of a priority. Ultimately, this was our main challenge as a duo. Communication was sometimes difficult, especially given our other classes and extracurriculars. However, we were able to overcome this obstacle by arranging times when we were both available. We also put an intentional effort into explaining our code to one another, as we sometimes completed different pieces of the project when working separately. Overall, the experience was beneficial in preparing us for the real world, and we believe that we worked well together. Below is a log that details our collaboration on a high level.

7. Log (roughly)

- Dinesh drives 30 minutes
- Amaan drives 1 hour
- Dinesh works alone for 2 hours
- Dinesh drives 1 hour and 30 minutes
- Amaan works alone for 1 hour
- Amaan drives 2 hours
- Dinesh drives 2 hours
- Dinesh works alone 2 hours
- Amaan works alone 2 hours

8. Brief explanation of custom critter

Nalliah is built around the idea that threats are scanned before anything else, every turn. Lines 1-3 check the square directly ahead and both forward diagonals, so the critter never walks past a target it could have taken, and an enemy one turn from being in front of us gets faced immediately (because that enemy is also one turn from infecting us). Hunger then decides how to attack. While satisfied, infecting is strictly better, since it removes an enemy and adds an ally,

increasing the population by two rather than one. Once hungry, eating is better, since it clears the hunger counter and grants the temporary burst, and a critter reduced to acting every third tick loses pretty easily. When nothing is in reach, Nalliah walks using `r1` as a countdown with five steps forward, then a 45-degree turn chosen at random to create some unpredictable patterns. The design is a bit flawed in that nothing watches the rear, and turning toward a diagonal enemy presents our front to something that may act first, in exchange for being relatively brief and easy to implement in code. It could definitely be refined with more time.